

Periodos Individuales por Tipo de Pasantía — Análisis de Alternativas

Fecha: 2026-06-18 **Estado:** Análisis con datos reales del cliente — requiere revisión del equipo **Contexto:** SDD Phase — Post-exploración, pre-propuesta formal

1. Resumen del Problema

Actualmente `t_internships_period` tiene un único par (`START_DATE`, `END_DATE`) para todo el periodo académico. El cliente solicita que cada tipo de pasantía (única/ingeniería, hospitalaria, comunitaria) tenga **fechas de inicio y fin independientes** dentro de un mismo periodo.

Datos reales proporcionados por el cliente:

Tipo	Inicio	Fin	Duración	Semanas
ING (ÚNICA)	23 marzo	19 junio	88 días	~12.5
HOSPITALARIA	16 marzo	8 mayo	53 días	~7.5
COMUNITARIA	18 mayo	3 julio	46 días	~6.5

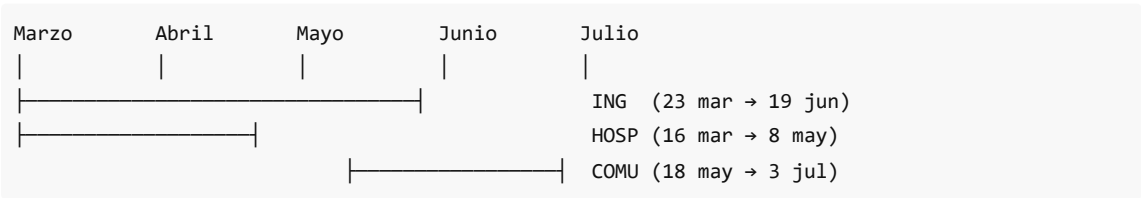
⚠ Los periodos actuales en seed data (`2025-I` , `2025-II` , `2026-I` , `2026-II`) son datos de ejemplo/placeholder, no representan la realidad académica. El diseño debe basarse en los datos del cliente.

¿Por qué es complejo?

- Afecta BD, backend, frontend y al menos 3 módulos (periods, pre-enrollment, enrollment)
- Involucra validaciones cruzadas con `t_career_internship_type` — no todas las carreras usan todos los tipos
- El estado del periodo (`PERIOD_STATUS`) y los grace days pueden o no heredarse por tipo
- Hay 24+ features en el sistema; el cambio es transversal

2. Patrón Identificado a Partir de los Datos del Cliente

2.1 — Línea de Tiempo Real



2.2 — Secuencia y Solapamiento

Relación	Desde	Hasta	Duración
HOSP ↔ ING (solapan)	23 mar	8 may	~6.5 semanas
HOSP ↔ COMU (NO solapan)	—	—	Buffer de 10 días

ING ↔ COMU (solapan)	18 may	19 jun	~4.5 semanas
----------------------	--------	--------	--------------

2.3 — Conclusiones del Patrón

1. **HOSPITALARIA es la más corta e intensiva** (~7.5 semanas) y arranca primero. Tiene sentido: las rotaciones hospitalarias requieren bloques concentrados.
2. **COMUNITARIA arranca 10 días después de que HOSP termina** — hay un buffer para cierre de evaluaciones. Para Enfermería (que tiene ambos tipos), son **secuenciales**: primero HOSP, luego COMU.
3. **ING/ÚNICA es la más larga** (~12.5 semanas) y cubre casi todo el periodo. Solapa con HOSP al inicio y COMU al final, pero como son carreras distintas, **no hay conflicto**.
4. **La prioridad del tipo (ÚNICA=0, HOSP=1, COMU=2) define el orden**, no la importancia. HOSP va antes, COMU después.
5. **Cada tipo dentro del mismo periodo tiene fechas independientes** — no comparten inicio ni fin.

3. Alternativas para el Modelo de Datos

Alternativa A — Tabla hija `t_period_type_dates`

Crear una nueva tabla que relacione periodo + tipo de pasantía con fechas específicas.

```
CREATE TABLE "t_period_type_dates" (
  "PERIOD_TYPE_DATE_ID" SERIAL NOT NULL PRIMARY KEY,
  "PERIOD_ID" INTEGER NOT NULL REFERENCES "t_internships_period"("PERIOD_ID"),
  "INTERNSHIP_TYPE_ID" INTEGER NOT NULL REFERENCES "t_internship_type"
("INTERNSHIP_TYPE_ID"),
  "START_DATE" DATE NOT NULL,
  "END_DATE" DATE NOT NULL,
  "ENROLLMENT_GRACE_DAYS" SMALLINT DEFAULT 21,
  "EVALUATION_GRACE_DAYS" SMALLINT DEFAULT 10,
  "CREATION_DATE" TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE("PERIOD_ID", "INTERNSHIP_TYPE_ID")
);
```

Pro	Contra
Integridad referencial real	JOIN extra en consultas
Indexable, consultas rápidas	Una tabla nueva que migrar
Per-type grace days natural	Más código CRUD
Escalable — agregar tipos no requiere migration	

Veredicto: Sólida. Funciona para el patrón identificado. Las fechas NOT NULL aseguran que cada tipo tenga fechas explícitas.

Alternativa B — JSONB en `t_internships_period`

Agregar una columna `TYPE_DATES` JSONB :

```
{
  "2": {"start": "2026-03-16", "end": "2026-05-08", "enrollment_grace_days": 15},
  "3": {"start": "2026-05-18", "end": "2026-07-03", "enrollment_grace_days": 10}
}
```

Pro	Contra
Sin tabla nueva, sin JOINS	Sin integridad referencial
Migración mínima — agregar columna	Validaciones en backend, no en BD
Escalable a nuevos tipos	Consultas tipo "periodos activos para tipo X" requieren <code>jsonb_extract_path</code>
Rápido de implementar	Indexar fechas dentro de JSONB es más lento y verboso
	No hay UNIQUE semántico real

Veredicto: viable para equipos pequeños con bajo volumen. En un sistema con 24 features y reportes cruzados, la falta de integridad referencial se vuelve deuda técnica.

Alternativa C — `t_period_internship_type` (tabla puente con atributos) — RECOMENDADA

Similar a la A, pero renombrada para reflejar que modela la **relación** periodo↔tipo como entidad de negocio, no como detalle técnico. Incluye `PERIOD_STATUS` para permitir estado independiente por tipo.

```
CREATE TABLE "t_period_internship_type" (
  "PERIOD_INTERNSHIP_TYPE_ID" SERIAL NOT NULL PRIMARY KEY,
  "PERIOD_ID" INTEGER NOT NULL REFERENCES "t_internships_period"
("PERIOD_ID"),
  "INTERNSHIP_TYPE_ID" INTEGER NOT NULL REFERENCES "t_internship_type"
("INTERNSHIP_TYPE_ID"),
  "START_DATE" DATE NOT NULL,
  "END_DATE" DATE NOT NULL,
  "ENROLLMENT_GRACE_DAYS" SMALLINT DEFAULT 21,
  "EVALUATION_GRACE_DAYS" SMALLINT DEFAULT 10,
  "PERIOD_STATUS" VARCHAR(45) NOT NULL DEFAULT '1',
  UNIQUE("PERIOD_ID", "INTERNSHIP_TYPE_ID")
);
```

Ejemplo con datos del cliente:

```
PERIOD_ID=10 (DESCRIPTION="2026-I")
├─ INT_TYPE_ID=2 (HOSP) → START=2026-03-16, END=2026-05-08, STATUS='2' (En Curso)
├─ INT_TYPE_ID=1 (ÚNICA) → START=2026-03-23, END=2026-06-19, STATUS='2' (En Curso)
└─ INT_TYPE_ID=3 (COMU) → START=2026-05-18, END=2026-07-03, STATUS='1' (Pendiente)
```

Pro	Contra
-----	--------

Modela explícitamente una relación de negocio	Misma complejidad que Alternativa A
PERIOD_STATUS por tipo — cada tipo tiene su ciclo de vida	Más columnas que A
Sigue el mismo patrón que t_career_internship_type (ya existe en el sistema)	
Escalable: nuevos tipos = nuevas filas, no nuevas columnas	

Diferencia clave con A: incluye PERIOD_STATUS como columna obligatoria — esto permite que cada tipo dentro del periodo tenga su propio ciclo de vida (Pendiente → En Curso → Culminado) de forma independiente.

Alternativa D — Periodos físicamente separados por tipo

Tres filas distintas en t_internships_period , agrupadas lógicamente:

```
PERIOD_GROUP: "2026-I" (nuevo campo opcional)
├─ PERIOD_ID=10 → "2026-I / ÚNICA"          (type_id=1, 23-mar→19-jun)
├─ PERIOD_ID=11 → "2026-I / HOSPITALARIA" (type_id=2, 16-mar→8-may)
└─ PERIOD_ID=12 → "2026-I / COMUNITARIA"  (type_id=3, 18-may→3-jul)
```

Pro	Contra
El código CRUD actual funciona casi sin cambios	Rompe el concepto semántico de "período académico"
Mínimo cambio en backend	Pre-inscripción apunta a PERIOD_ID — habría que resolver cuál de los 3
Cada tipo tiene su propio estado natural	Reportes "qué pasó en 2026-I" requieren agrupar
	La descripción se vuelve ambigua en UI
	Carreras con un solo tipo duplican lógica

Veredicto: tentador por bajo esfuerzo aparente, pero le pega al modelo de dominio donde un período agrupa varios tipos. No recomendado.

Alternativa E — EAV (Entity-Attribute-Value) t_period_config

Tabla genérica de configuraciones:

```
CREATE TABLE "t_period_config" (
  "PERIOD_CONFIG_ID" SERIAL PRIMARY KEY,
  "PERIOD_ID"         INTEGER NOT NULL REFERENCES "t_internships_period"("PERIOD_ID"),
  "INTERNSHIP_TYPE_ID" INTEGER REFERENCES "t_internship_type"("INTERNSHIP_TYPE_ID"),
  "CONFIG_KEY"        VARCHAR(50) NOT NULL,
  "CONFIG_VALUE"      VARCHAR(255) NOT NULL,
  UNIQUE("PERIOD_ID", "INTERNSHIP_TYPE_ID", "CONFIG_KEY")
);
```

Pro	Contra
Máxima flexibilidad — cualquier configuración futura	Las consultas requieren PIVOT → lentas y complejas
Una tabla sirve para fechas, configs, lo que sea	Sin tipado de datos, sin validación de fechas en BD
Cero migraciones futuras	Lógica de negocio dispersa y frágil
	Rendimiento pobre en reportes y dashboards

Veredicto: antipatrón para un dominio académico estable. Los tipos de pasantía y sus fechas son estructura conocida, no configuraciones dinámicas. No recomendado.

4. Incidencia del Factor Carrera

La tabla `t_career_internship_type` define qué tipos aplican a cada carrera:

Carrera	Tipos que aplican
TSU Enfermería (ID=3)	HOSPITALARIA + COMUNITARIA
Ing. Informática (ID=4)	ÚNICA
Ing. Agroindustrial (ID=5)	ÚNICA

Esto introduce restricciones y consideraciones de diseño:

4.1 — Experiencia del estudiante vs. Administración del periodo

Visión estudiante (Enfermería):

- Se pre-inscribe en HOSPITALARIA (mar 16 → may 8)
- Cuando HOSP termina, se pre-inscribe en COMUNITARIA (may 18 → jul 3)
- Para él/ella, son periodos secuenciales

Visión admin:

- Crea UN periodo "2026-I" con 3 bloques de fechas internos
- Cada tipo tiene su propio estado de avance

Esto significa que **el estudiante no necesita ver 3 tipos de periodo — ve su tipo asignado según la carrera.**

4.2 — Validación por carrera al crear periodo

Como las carreras ya están mapeadas a tipos en `t_career_internship_type` :

- Si se crea un periodo **sin fechas para HOSPITALARIA**, Enfermería no puede pre-inscribir estudiantes → el sistema debería al menos advertirlo.
- Si solo se definen fechas para ÚNICA, Ing. Informática e Ing. Agroindustrial pueden operar sin problemas.

Regla propuesta: Al crear un periodo, mostrar un resumen de "carreras afectadas por tipo" para que el admin sepa qué carreras se quedan sin fechas si omite algún tipo. Pero no bloquear — dejar que el admin decida.

4.3 — Reportes multi-carrera

Para consultas tipo "estudiantes activos en 2026-I", la lógica pasa de:

```
-- Antes: fecha actual dentro del periodo
WHERE NOW() BETWEEN START_DATE AND END_DATE
```

a:

```
-- Después: fecha actual dentro del tipo específico de cada práctica
WHERE NOW() BETWEEN pit.START_DATE AND pit.END_DATE
AND pit.INTERNSHIP_TYPE_ID = pp.INTERNSHIP_TYPE_ID
AND pit.PERIOD_ID = pp.PERIOD_ID
```

Esto impacta `reports.controller.ts` , `tracking.controller.ts` , y las consultas del dashboard.

5. Decisiones Pendientes (Casos Hipotéticos)

Caso 1 — Estado del Periodo: ¿Compartido o por Tipo?

Escenario real: Enfermería, mismo periodo.

- HOSPITALARIA: 16 mar → 8 may (en curso)
- COMUNITARIA: 18 may → 3 jul (aún no arranca, falta 1 mes)

Si el estado es...	Comportamiento
Compartido	PERIOD_STATUS del periodo = "En Curso" (porque HOSP está activa). COMU aparece como "no disponible" aunque el periodo esté activo.
Por tipo	PERIOD_STATUS de HOSP = "En Curso", PERIOD_STATUS de COMU = "Pendiente". Cada tipo tiene su ciclo.

● **Impacto:** Los datos del cliente muestran que HOSP y COMU NO solapan — son secuenciales. Si COMU no empieza hasta que HOSP termina, **tener estado por tipo es casi obligatorio** para que los estudiantes de Enfermería vean correctamente cuándo pueden inscribirse en COMU.

Caso 2 — Grace Days: ¿Por Periodo o por Tipo?

Actualmente `ENROLLMENT_GRACE_DAYS` (21) y `EVALUATION_GRACE_DAYS` (10) están en `t_internships_period`.

Si los grace days son...	Comortamiento
Heredados del periodo	Todos los tipos comparten los mismos grace days. Más simple, no requiere migrar migraciones existentes.
Por tipo	Cada tipo tiene su propia ventana. HOSP podría tener 15 días de gracia, COMU 21. Flexible pero más complejo.

● **Impacto:** Con el patrón secuencial HOSP→COMU, probablemente necesitan grace days distintos: HOSP tiene evaluación antes (porque termina antes), COMU después. **Se recomienda por tipo**, pero puede ser postergado a una segunda iteración si se quiere reducir alcance inicial.

Caso 3 — ¿Qué pasa si el admin crea un periodo sin fechas para un tipo?

Escenario: Solo se definen fechas para ÚNICA y HOSPITALARIA, pero no para COMUNITARIA.

Opción	Comportamiento
Bloquear	No se puede crear/guardar el periodo si faltan fechas para algún tipo activo en t_career_internship_type.
Advertencia suave	Se guarda, pero muestra: "Tipos sin fechas: COMUNITARIA — las carreras TSU Enfermería no podrán pre-inscribirse en este tipo."
Validación dinámica	Solo exige fechas para tipos que tengan al menos un estudiante próximo a inscribirse ese periodo.

Caso 4 — ¿Permitimos solapamiento entre tipos dentro del mismo periodo?

Escenario real: ING (23 mar → 19 jun) y COMU (18 may → 3 jul) solapan del 18 may al 19 jun.

Opción	Comportamiento
Permitir	Los tipos son independientes, cada uno tiene su cronograma.
Bloquear	No se permite que dos tipos tengan fechas superpuestas.

✅ **Respuesta clara:** Los datos del cliente muestran solapamiento ING↔COMU. **Debe permitirse.** La validación actual de `periodValidations.ts` (que rechaza cualquier solapamiento) necesita modificarse para ignorar solapamientos entre tipos distintos.

Caso 5 — Edición de fechas con prácticas activas

Escenario: HOSPITALARIA en curso con 30 estudiantes. El admin necesita extender la fecha de fin.

Opción	Comportamiento
Permitir siempre	El admin cambia las fechas. Las prácticas existentes mantienen sus START_DATE/END_DATE porque se copiaron al pre-inscribir.
Solo extender	Solo permitir mover END_DATE hacia adelante, nunca acortar.
Con auditoría	Se permite pero se registra en t_change_log con motivo obligatorio.

Caso 6 — Duración mínima por tipo

Hoy se valida duración mínima de 16 semanas en el periodo general. Con los datos del cliente:

Tipo	Duración	¿Pasa 16 semanas?
ING	~12.5 semanas	✗
HOSP	~7.5 semanas	✗
COMU	~6.5 semanas	✗

● **Impacto:** la validación de 16 semanas **no aplica para tipos individuales**. Hay que relajarla o redefinirla para que valide la suma del periodo completo (HOSP + buffer + COMU = ~16 semanas para Enfermería), no cada tipo por separado.

6. Recomendaciones del Análisis

6.1 — Modelo de datos recomendado

Alternativa C — `t_period_internship_type` (tabla puente con atributos)

Basado en los datos reales del cliente:

1. **El estado POR TIPO es necesario** porque los tipos tienen ciclos de vida independientes. HOSP puede estar "En Curso" mientras COMU está "Pendiente". La columna `PERIOD_STATUS` en la tabla puente lo resuelve sin tocar `t_internships_period`.
2. **Sigue el patrón existente:** `t_career_internship_type` ya es una tabla puente en el sistema. Misma convención de nomenclatura, mismo approach.
3. **Las fechas NOT NULL** evitan ambigüedades — cada tipo en cada periodo tiene fechas explícitas.
4. **Sin fallback al periodo padre** — cada tipo es independiente y se define por sí mismo.
5. **Seed data actual es placeholder** — no hay datos reales que migrar, solo estructura. La migración puede ser limpia.

6.2 — Proceso de implementación sugerido

FASE 0 — Fundación BD

- └─ Migration: crear `t_period_internship_type`
- └─ Seed: poblar con los 3 tipos para cada periodo existente
- └─ Rollback script listo

FASE 1 — Backend

- └─ Actualizar `periods.controller.ts` (CRUD + fechas por tipo)
- └─ Actualizar `period-validator.middleware.ts` (resolver fechas por `PERIOD_ID + INTERNSHIP_TYPE_ID`)
- └─ Actualizar `pre-enrollments.controller.ts` (copiar fechas del tipo específico)
- └─ Actualizar `enrollments.controller.ts`
- └─ Actualizar consultas en `reports/tracking/dashboard`

FASE 2 — Frontend

- └─ Rediseñar `PeriodModal.tsx` (pares de fecha por tipo visibles)
- └─ Actualizar `PeriodTable.tsx` (progreso calculado por tipo)
- └─ Actualizar `periodUtils.ts` y `periodValidations.ts`
- └─ Verificar `PreEnrollmentModal.tsx` y `EnrollmentModal.tsx`

FASE 3 — Verificación

- └─ Test unitarios backend (nuevos escenarios de fechas por tipo)
- └─ Test frontend (validaciones Zod actualizadas)
- └─ Pruebas en BD local
- └─ Verificación de reportes

FASE 4 – Migración a producción

└─ Previa aprobación de todas las pruebas locales

6.3 — Resumen de decisiones requeridas del equipo

#	Decisión	Opción recomendada	¿Bloqueante para arrancar?
1	Estado del periodo	Por tipo (cada tipo tiene su <code>PERIOD_STATUS</code>)	✓ Sí
2	Grace days	Por tipo (ideal), o heredados (alcance reducido)	✗ No (puede ser post-MVP)
3	Cobertura al crear periodo	Advertencia suave — informar pero no bloquear	✗ No
4	Solapamiento intra-periodo	Permitido — los tipos son independientes	✓ Sí
5	Edición con prácticas activas	Solo extender con registro en audit log	✗ No
6	Duración mínima	Por periodo completo, no por tipo (suma de tipos + buffer)	✓ Sí
7	Etiqueta del periodo	Ej: "2026-I" como descriptor general; los tipos son internos	✗ No

7. Próximos Pasos

- ✓ Revisar este documento con el equipo
- Responder las 7 decisiones (sección 6.3)
- Formalizar propuesta SDD (`/sdd-propose`) con las decisiones tomadas
- Configurar entorno local de pruebas BD (copia de seguridad para testing)
- Especificación técnica (`/sdd-spec`)
- Diseño técnico (`/sdd-design`)
- Plan de implementación con tareas (`/sdd-tasks`)
- Implementar en local → verificar → migrar a producción

Documento generado como parte del flujo SDD — fase de exploración. Los datos de fechas fueron proporcionados directamente por el cliente como parte del requerimiento. Next: `/sdd-propose` cuando las decisiones del equipo estén tomadas.